HelixVPN — Phase 1 MVP Build Specification

HelixVPN — Phase 1 MVP Build Specification

Revision: 2 **Last modified:** 2026-07-04T12:00:00Z **Status:** primary research source — elaborated + made authoritative by `../final/07-phase1-mvp-wbs.md` (the executable WBS with `HVPN-P1-NNN` work items, acceptance criteria, and captured-evidence requirements) and `../final/02-control-plane.md` (the authoritative DDL/protobuf/API surface). Where this document and `final/07-phase1-mvp-wbs.md` disagree, **`final/` wins** per `SPECIFICATION.md` §12 versioning discipline — this document is preserved as the cited primary source ([04_P1] throughout `final/`), not re-derived.

Companion to: `HelixVPN-Architecture-Refined.md` (the *what*) and `HelixVPN-Phase0-Spike.md` (the *prove-it-first*). **Entry condition:** Phase 0 gates G1–G6 cleared (the data path, MASQUE, iOS memory, edge language, FFI, and file-based reconciliation all proven). **Goal of Phase 1:** turn the proven slice into a **self-hostable MVP** — a real Go control plane that streams desired-state to many agents, a durable data model, an event-driven backbone, a working policy engine, the three apps in their first shippable form, and a one-command `podman` deploy.

The discipline of Phase 1: **the static `map.json` becomes a streamed protobuf `NetworkMap`, and the throwaway interfaces from Phase 0 acquire durability, identity, and policy — without changing their shape.**

---

0. Scope delta (Phase 0 → Phase 1)

Concern	Phase 0 (spike)	Phase 1 (MVP)
Desired-state source	static `map.json` file	

Concern	Phase 0 (spike)	Phase 1 (MVP)
		Go `coordinator` streaming `WatchNetworkMap` over Connect/gRPC
Identity	none (hardcoded keys)	OIDC + anonymous device tokens; per-device enrollment + certs
Persistence	none	Postgres + RLS (truth) and Redis (ephemeral presence/events)
Events	file-watch	Redis Streams with consumer groups + dead-letter
Policy	"allow all"	declarative ACL model + compiler → per-node peer/verdict sets
Transports	plain-UDP + MASQUE	+ LWO; auto-escalation ladder driven by handshake events
Apps	Flutter-Linux toggle	Access (iOS/Android/Linux), Connector daemon, Console (web)
Deploy	`make spike` on one box	`helixvpnctl` + Podman quadlets, single-node self-host
Privacy posture	n/a	kill-switch, DNS-leak protection, **no durable connection logs**

Out of scope for Phase 1 (→ Phase 2): Shadowsocks/UoT, DAITA shaping, multi-hop, PQ handshake, HA/multi-region, HarmonyOS/Aurora, billing, the WASM web tunnel. MVP runs as a single control-plane binary against one Postgres + one Redis.

---

1. Control-plane architecture (modular monolith)

One Go binary, many packages, deployable as one container. Package boundaries == future service boundaries (so the split in Phase 2+ is mechanical, not a rewrite).

```
helix-go/
├── cmd/
│   ├── helixd/           # the control-plane binary (all modules wired together)
│   └── helixvpnctl/      # bootstrap + ops CLI (Cobra)
├── internal/
│   ├── identity/         # tenants, users, OIDC, device tokens
```

```
│   ├── registry/         # devices, connectors, prefixes,
presence
│   ├── ipam/             # overlay IP allocation
│   ├── pki/              # WG key registry, device certs,
rotation
│   ├── policy/           # ACL model + compiler
│   ├── coordinator/      # builds NetworkMaps, computes deltas,
streams to agents
│   ├── events/           # Redis Streams producer/consumer
abstraction
│   ├── telemetry/        # Prometheus metrics, control-action
audit
│   ├── api/              # Gin REST (apps) + Connect handlers
(agents) + WS/SSE
│   └── store/            # Postgres access, RLS session helpers,
migrations
├── proto/                # *.proto (agent contracts) → buf
generate
└── openapi/              # REST contract → generated Dart/TS
clients
```

Wiring rule: modules talk through **interfaces + events**, never by importing each other's stores. `coordinator` reads `registry`/`policy`/`pki` through interfaces and reacts to events; it owns no tables of its own except a cache.

---

2. Data model — Postgres DDL with Row-Level Security

The durable store holds **identity, topology, and policy** — never connection or traffic logs (the no-logging promise is enforced here by *absence* and by a CI lint, §11.4).

2.1 Core DDL

```sql
-- ============ tenancy & identity ============
CREATE TABLE tenants (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name        text NOT NULL,
  created_at  timestamptz NOT NULL DEFAULT now()
);

CREATE TYPE user_role AS ENUM ('admin','operator','member');

CREATE TABLE users (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id) ON DELETE
        CASCADE,
```

```
  email       text,                       -- nullable: anonymous
        device-token users
  oidc_sub    text,
        -- nullable: when SSO is used
  role        user_role NOT NULL DEFAULT 'member',
  created_at  timestamptz NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, email),
  UNIQUE (tenant_id, oidc_sub)
);

-- ============ devices (clients AND connectors) ============
CREATE TYPE device_kind AS ENUM ('client','connector');

CREATE TABLE devices (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id     uuid NOT NULL REFERENCES tenants(id) ON DELETE
        CASCADE,
  user_id       uuid REFERENCES users(id) ON DELETE SET NULL,
  kind          device_kind NOT NULL,
  name          text NOT NULL,
  wg_pubkey     bytea NOT NULL,           -- 32-byte Curve25519
        public key
  overlay_ip    inet NOT NULL,
        -- allocated from tenant ULA /48
  os            text,                     -- ios|android|linux|
        windows|macos|harmonyos|aurora
  enrolled_at   timestamptz NOT NULL DEFAULT now(),
  last_seen_at  timestamptz,              -- coarse; refreshed
        from presence, not per-packet
  revoked_at    timestamptz,
  UNIQUE (tenant_id, wg_pubkey),
  UNIQUE (tenant_id, overlay_ip)
);

CREATE TABLE connectors (
  device_id   uuid PRIMARY KEY REFERENCES devices(id) ON DELETE
        CASCADE,
  tenant_id   uuid NOT NULL REFERENCES tenants(id) ON DELETE
        CASCADE,
  site_name   text NOT NULL
);

CREATE TABLE advertised_prefixes (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id     uuid NOT NULL REFERENCES tenants(id) ON DELETE
        CASCADE,
  connector_id  uuid NOT NULL REFERENCES connectors(device_id) ON
        DELETE CASCADE,
  cidr          cidr NOT NULL,
  enabled       boolean NOT NULL DEFAULT true,
  created_at    timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX ON advertised_prefixes (tenant_id, connector_id);
```

```
-- ============ groups (for policy) ============
CREATE TABLE groups (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id) ON DELETE
        CASCADE,
  name        text NOT NULL,
  UNIQUE (tenant_id, name)
);
CREATE TABLE group_members (
  group_id    uuid NOT NULL REFERENCES groups(id) ON DELETE
        CASCADE,
  device_id   uuid NOT NULL REFERENCES devices(id) ON DELETE
        CASCADE,
  PRIMARY KEY (group_id, device_id)
);

-- ============ policy (source + compiled) ============
CREATE TABLE policies (
  id           uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id    uuid NOT NULL REFERENCES tenants(id) ON DELETE
        CASCADE,
  spec         jsonb NOT NULL,            -- the declarative ACL
        document (§5)
  version      bigint NOT NULL,           -- monotonic per tenant
  compiled_at  timestamptz,
  created_at   timestamptz NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, version)
);

-- ============ overlay IPAM ============
CREATE TABLE overlay_pools (
  tenant_id   uuid PRIMARY KEY REFERENCES tenants(id) ON DELETE
        CASCADE,
  ula_prefix  cidr NOT NULL,             -- e.g.
        fd7a:helix:<rand>::/48
  next_host   bigint NOT NULL DEFAULT 2   -- ::1 reserved for
        gateway
);

-- ============ PKI ============
CREATE TABLE device_certs (
  id           uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id    uuid NOT NULL REFERENCES tenants(id) ON DELETE
        CASCADE,
  device_id    uuid NOT NULL REFERENCES devices(id) ON DELETE
        CASCADE,
  serial       text NOT NULL,
  not_after    timestamptz NOT NULL,
  revoked      boolean NOT NULL DEFAULT false,
  created_at   timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX ON device_certs (tenant_id, device_id) WHERE NOT
        revoked;
```

```
-- ============ audit (control actions only — never traffic)
        ============
CREATE TABLE audit_events (
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  tenant_id   uuid NOT NULL REFERENCES tenants(id) ON DELETE
        CASCADE,
  actor       text NOT NULL,              -- user id / "system"
  action      text NOT NULL,              -- e.g.
        "device.revoke","policy.update"
  target      text,
  ts          timestamptz NOT NULL DEFAULT now(),
  meta        jsonb
);
CREATE INDEX ON audit_events (tenant_id, ts DESC);
```

2.2 Row-Level Security (tenant isolation at the database)

Every tenant-scoped table gets the same treatment. The app sets the tenant per transaction; Postgres enforces isolation even if a query forgets a `WHERE`.

```
ALTER TABLE devices ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON devices
  USING       (tenant_id = current_setting('app.tenant_id')::uuid)
  WITH CHECK  (tenant_id =
        current_setting('app.tenant_id')::uuid);
-- repeat for: users, connectors, advertised_prefixes, groups,
        group_members,
--              policies, overlay_pools, device_certs, audit_events
```

```
// store: every request runs inside a tenant-scoped tx
func (s *Store) WithTenant(ctx context.Context, tenantID
        uuid.UUID,
    fn func(q *Queries) error) error {
    tx, _ := s.db.BeginTx(ctx, nil)
    // SET LOCAL is transaction-scoped; cannot leak across pooled
        connections
    if _, err := tx.ExecContext(ctx,
        "SELECT set_config('app.tenant_id', $1, true)",
        tenantID.String()); err != nil {
        _ = tx.Rollback(); return err
    }
    if err := fn(New(tx)); err != nil { _ = tx.Rollback(); return
        err }
    return tx.Commit()
}
```

Use a **non-superuser DB role** for the app (superusers bypass RLS). Migrations via `goose`/`atlas`; queries via `sqlc` (compile-time-checked SQL → Go), which pairs cleanly with the typed `Queries` above.

---

3. Agent contract — protobuf (`WatchNetworkMap` and friends)

This is the static `map.json` from Phase 0, now a streamed, versioned, delta-capable protobuf served over **Connect** (works as gRPC, gRPC-Web, and the Connect protocol — so the *same* service serves native agents over HTTP/2 and browser clients over HTTP/1.1).

```
// proto/helix/agent/v1/agent.proto
syntax = "proto3";
package helix.agent.v1;

service Coordinator {
  // One-time device enrollment (exchanges an enroll token for
        identity + cert).
  rpc Enroll(EnrollRequest) returns (EnrollResponse);

  // The spine of the system: open once, get a snapshot, then a
        delta stream.
  rpc WatchNetworkMap(WatchRequest) returns (stream MapUpdate);

  // Connectors push their advertised prefixes (also settable via
        Console/API).
  rpc AdvertisePrefixes(AdvertiseRequest) returns
        (AdvertiseResponse);

  // Lightweight heartbeat / status (presence, current transport,
        rtt). No traffic data.
  rpc ReportStatus(StatusReport) returns (StatusAck);
}

message EnrollRequest {
  string enroll_token = 1;     // issued by Console/identity,
        short-lived
  bytes  wg_pubkey    = 2;     // device-generated, private key
        never leaves device
  string os           = 3;
  string name         = 4;
  DeviceKind kind     = 5;
}
enum DeviceKind { DEVICE_KIND_UNSPECIFIED = 0; CLIENT = 1;
        CONNECTOR = 2; }

message EnrollResponse {
  string device_id   = 1;
  string overlay_ip  = 2;      // allocated by IPAM, e.g.
        "fd7a:helix:1::2"
  bytes  device_cert = 3;      // short-lived mTLS cert for the
        control channel
  GatewayInfo gateway = 4;
}
```

```
message WatchRequest {
  string device_id   = 1;
  uint64 known_version = 2;    // 0 => send full snapshot; else
        send deltas since
}

// Either a full snapshot or an incremental delta. Agents
        reconcile to it.
message MapUpdate {
  uint64 version = 1;
  oneof body {
    NetworkMap snapshot = 2;
    MapDelta   delta    = 3;
    KeepAlive  keepalive = 4;  // periodic, proves liveness
        without state churn
  }
}

message NetworkMap {
  Node          self     = 1;
  GatewayInfo   gateway  = 2;
  repeated Peer peers    = 3;  // peers this node may reach
        (already policy-filtered)
  DnsConfig     dns      = 4;
  TransportPolicy transport = 5;
}

message Node   { string device_id = 1; string overlay_ip = 2; }
message GatewayInfo {
  string endpoint   = 1;       // "gw.example:443"
  bytes  wg_pubkey  = 2;
  string masque_sni = 3;       // host to present for MASQUE/
        HTTP-3 masquerade
}
message Peer {
  string device_id      = 1;
  bytes  wg_pubkey       = 2;
  repeated string allowed_ips = 3;  // compiled from policy: only
        what this node may reach
  string endpoint        = 4;       // usually via gateway relay
        in MVP
  bool   is_connector    = 5;
}
message DnsConfig { repeated string resolvers = 1; repeated
        string search = 2; }

message TransportPolicy {
  // ordered escalation ladder the client walks on handshake
        failure
  repeated string order = 1;   // ["plain-udp","lwo","masque-h3"]
  bool allow_user_override = 2;
}
```

```
message MapDelta {
  repeated Peer   upsert_peers = 1;
  repeated string remove_peer_ids = 2;
  TransportPolicy transport   = 3;  // present if changed
  DnsConfig       dns         = 4;  // present if changed
}

message AdvertiseRequest { string device_id = 1; repeated string
        cidrs = 2; }
message AdvertiseResponse { bool accepted = 1; repeated string
        conflicts = 2; }

message StatusReport {
  string device_id = 1;
  string transport = 2;        // current transport in use
  uint32 rtt_ms    = 3;
  // deliberately NO bytes/flows/destinations — presence + health
        only
}
message StatusAck {}
message KeepAlive {}
```

Key properties: `known_version` makes reconnects cheap (resume from a version, no full resync); the server sends a **snapshot then deltas**; peers arrive **already policy-filtered** so the client never learns about nodes it can't reach (need-to-know, privacy-preserving). This protobuf is the literal evolution of Phase 0's `map.json` — same fields, now streamed and authorized.

---

4. The `coordinator` — building and streaming maps

The coordinator is the brain. It owns no durable tables; it **reacts to events**, recomputes affected maps, and pushes minimal deltas down open streams.

4.1 Responsibilities

1. Maintain an in-memory, per-tenant **topology graph** (devices, connectors, prefixes, groups, compiled policy), hydrated from Postgres on boot and kept fresh by events.
2. Hold the set of **open `WatchNetworkMap` streams** (one per connected agent).
3. On each relevant event: determine the **affected agents**, compute each one's **delta**, push it. Convergence target: **p99 < 1 s** from event to delta-on-wire (a measured SLO, §10).

4.2 Map computation (per node)

```
buildMap(node):
  reachable := policy.PeersVisibleTo(node)         # need-to-know
filter
  peers := []
  for p in reachable:
     allowed := policy.AllowedIPs(node -> p)        # compiled
CIDRs+ports
     peers.append(Peer{p.pubkey, allowed, relayEndpoint(p),
p.isConnector})
  return NetworkMap{self, gateway, peers, dns(node),
transportPolicy(node)}
```

`relayEndpoint` in MVP always routes peer traffic **through the gateway** (hub-and-spoke); direct peer-to-peer is a Phase 2 optimization. This keeps the MVP's data path identical to the proven Phase 0 slice.

4.3 Streaming + delta loop (Go sketch)

```go
func (c *Coordinator) WatchNetworkMap(ctx context.Context,
    req *connect.Request[agentv1.WatchRequest],
    stream *connect.ServerStream[agentv1.MapUpdate]) error {

    dev := authDevice(ctx)                       // from mTLS
        device cert
    sub := c.subscribe(dev.TenantID, dev.ID)     // registers
        stream + presence
    defer sub.Close()

    // 1. snapshot (or resume from known_version)
    m := c.buildMap(dev)
    _ = stream.Send(&agentv1.MapUpdate{Version:
        c.version(dev.TenantID),
        Body: &agentv1.MapUpdate_Snapshot{Snapshot: m}})

    // 2. delta loop: events arrive on sub.C, keepalive on a
        ticker
    ka := time.NewTicker(20 * time.Second); defer ka.Stop()
    for {
        select {
        case <-ctx.Done():        return ctx.Err()
        case d := <-sub.C:        _ = stream.Send(deltaUpdate(d))
        case <-ka.C:              _ = stream.Send(keepAlive())
        }
    }
}
```

4.4 Fan-out: events → affected agents

The coordinator consumes the event streams (§5), updates its topology graph, then computes which open streams are affected and enqueues deltas to each. Example: a `policy.compiled` event recomputes visibility for the whole tenant; a `connector.prefixes.changed` event only touches nodes whose policy grants that connector. Compute the **minimal** affected set — never broadcast a full resync on a small change.

---

5. Event backbone — Redis Streams contracts

Redis Streams is the MVP bus (NATS JetStream is the Phase-2 scale swap; the envelope is bus-agnostic so it's a transport change only).

5.1 Streams & consumer groups

Stream	Producers	Consumer groups
`events:devices`	identity, registry	coordinator, telemetry, audit
`events:routes`	registry	coordinator, telemetry
`events:policy`	policy	coordinator, audit
`events:presence`	api (heartbeats)	coordinator (TTL/online state)
`events:gateway`	edge health probes	coordinator, telemetry

5.2 Envelope (every event)

```
{
  "id":         "<redis-stream-id>",
  "type":       "device.revoked",
  "tenant_id": "uuid",
  "ts":         "RFC3339",
  "actor":      "user-uuid|system",
  "payload":    { /* type-specific */ },
  "trace_id":   "for correlation"
}
```

5.3 Event types & payloads (the taxonomy made concrete)

Type	Payload	Coordinator reaction
`device.enrolled`	`{device_id, kind, overlay_ip}`	add node to graph; if policy grants, push to peers' maps
`device.online` / `device.offline`	`{device_id}`	update presence; peers see relay availability
`device.revoked`	`{device_id}`	remove node; push peer-removal delta to everyone who saw it; edge drops sessions
`connector.attached`	`{device_id, site}`	register connector
`connector.prefixes.changed`	`{connector_id, cidrs[]}`	recompute routes; push to nodes whose policy includes it
`route.conflict.detected`	`{cidr, connector_ids[]}`	flag for overlapping-CIDR handling (§7); surface in Console
`policy.updated`	`{version}`	trigger compile
`policy.compiled`	`{version}`	recompute tenant visibility; push deltas
`gateway.failover`	`{from, to}`	re-point affected nodes' gateway endpoint

5.4 Producer / consumer mechanics

```
// produce
xid, _ := rdb.XAdd(ctx, &redis.XAddArgs{Stream: "events:policy",
        Values: env}).Result()

// consume (durable group, at-least-once)
res, _ := rdb.XReadGroup(ctx, &redis.XReadGroupArgs{
    Group: "coordinator", Consumer: hostID,
    Streams: []string{"events:policy", ">"}, Count: 64, Block: 5
        * time.Second,
```

```
}).Result()
// ... handle, then XACK; reclaim stuck entries with XAUTOCLAIM
        (dead-letter recovery)
```

At-least-once delivery + **idempotent reactions** (deltas are computed from current graph state, so a replayed event is harmless). This is what replaces Phase 0's file-watch and the original doc's cron-restart loops.

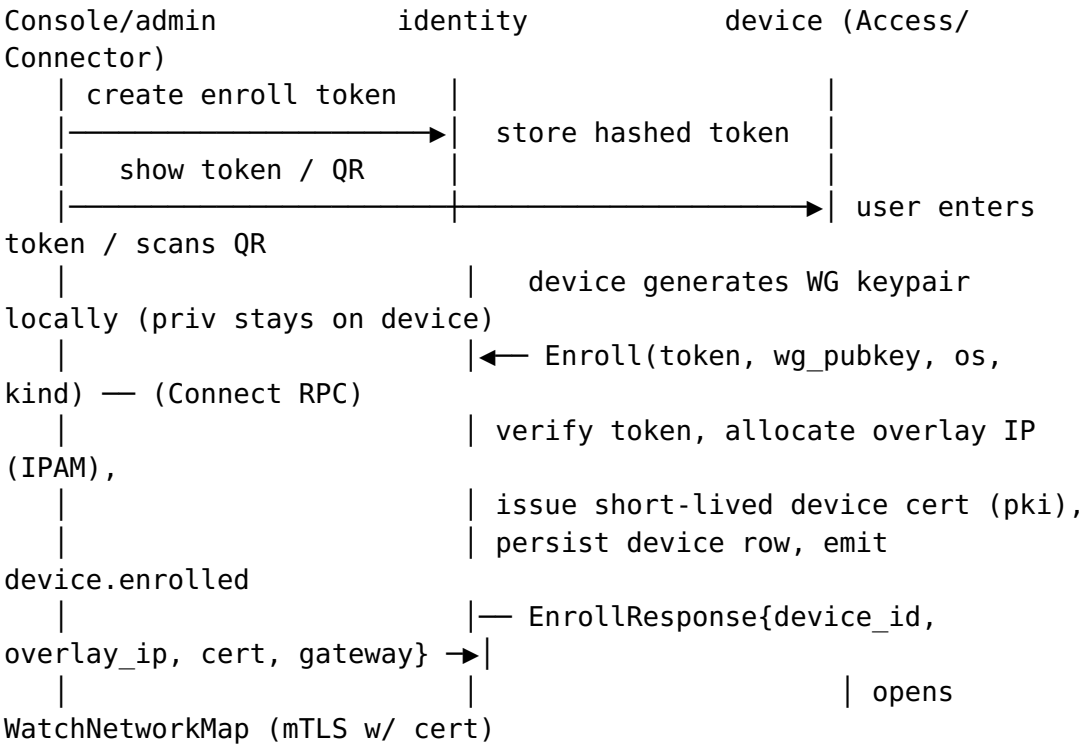
---

6. Identity, enrollment & PKI

6.1 Two identity modes (both MVP)

- **Managed (OIDC):** tenant admins log into the Console via OIDC (Keycloak/Authentik/any IdP). Users belong to a tenant; devices belong to users.
- **Anonymous (privacy mode):** a tenant can mint **device enroll tokens** with no email/SSO — the Mullvad "account number, no PII" posture. The device gets identity + cert without any personal data stored.

6.2 Enrollment flow (device never surrenders its private key)



6.3 PKI

- **WG keys:** device-generated; only the **public** key is registered. The control plane never sees private keys.
- **Control-channel mTLS:** `pki` issues a short-lived (e.g., 24h) device cert; the agent uses it to authenticate the `WatchNetworkMap`/RPC channel. Renewed automatically over the existing channel before expiry.
- **Rotation & revoke:** rotation is a re-issue; `device.revoked` immediately (a) drops the device's WG peer from every relevant map, (b) is enforced at the edge (kernel WG peer removed), (c) marks the cert revoked. Revocation latency target == convergence SLO (< 1 s).
- **Gateway root / KMS:** the tenant CA key is the one true secret; back it up (KMS or offline) — it and Postgres are the only stateful things to protect (per architecture §10).

---

7. Policy model & compiler

The original doc had "allow all." This is the real access-control brain. Model is **Tailscale-ACL-flavored**, declarative, default-deny.

7.1 Source document (stored in `policies.spec`)

```
{
  "groups": {
    "group:admins":      ["alice@corp", "bob@corp"],
    "group:contractors": ["carol@ext"]
  },
  "hosts": {
    "warehouse-cams": "10.10.0.0/24",      // served by connector
A
    "office-lan":     "192.168.50.0/24"    // served by connector
B
  },
  "acls": [
    { "action": "accept", "src": ["group:admins"],      "dst":
["*:*"] },
    { "action": "accept", "src": ["group:contractors"], "dst":
["warehouse-cams:554,80"] }
  ],
  "exitNodes": ["group:admins"]             // who may use the
gateway as a full-tunnel exit
}
```

7.2 Compilation algorithm

```
compile(tenant, spec) -> CompiledPolicy:
  resolve groups -> sets of device_ids (via users + group_members)
  resolve hosts  -> CIDRs (cross-check against
advertised_prefixes; emit route.conflict if ambiguous)
  for each device d:
     visible[d] = {}                          # peers d may reach
(need-to-know)
     for rule in acls where d in resolve(rule.src):
        for dst in rule.dst:
           targets = expand(dst)              # connector(s)
serving CIDR, or peer devices, or exit
           for t in targets:
              visible[d] += t
              allowedIPs[d][t] += (dst.cidr, dst.ports)
  return { visible, allowedIPs, exitNodes }
```

- Output is exactly what `coordinator.buildMap` consumes: per-node **visible peers + allowed IPs/ports**.
- Compiled to two artifacts: WireGuard **`AllowedIPs`** (coarse, per-peer CIDR) on the data path, plus a finer **port-level verdict map** enforced at the edge via nftables/eBPF (since WG `AllowedIPs` is CIDR-only, not port-aware).
- Compilation is **pure + versioned**: same spec ⇒ same output; bump `policies.version`, emit `policy.compiled`, coordinator diffs and pushes. A bad policy can be **rolled back** by activating a prior version (instant).

7.3 Validation (fail closed)

`policy.update` runs the compiler in dry-run first; reject on: unknown group/host, a `dst` CIDR not covered by any advertised prefix, or a rule that would grant a revoked device. Only a clean compile flips the active version.

---

8. API surface (Gin REST + Connect + WS/SSE)

Audience	Protocol	Examples
Apps (Access/ Connector/ Console)	**REST via Gin**	`POST /v1/enroll-tokens`, `GET /v1/devices`, `POST /v1/policies`, `GET /v1/networks`
Live UI	**WebSocket / SSE**	`GET /v1/stream` → `device.online`, `route.changed`, `handshake.failing`
Agents		`Coordinator.WatchNetworkMap`, `.Enroll`,`.AdvertisePre`

Audience	Protocol	Examples
	Connect/ gRPC	

- One server, multiplexed: Gin for REST/WS on HTTP, Connect handlers mounted alongside (Connect speaks HTTP/2 for native agents and downgrades for browsers — so a future WASM Console can call the same `Coordinator` service).
- **Authz:** REST uses OIDC/session or API token + RBAC (`admin`/`operator`/`member`); agent RPCs use the device mTLS cert. RLS underneath is the backstop.
- Contracts generated: OpenAPI → Dart/TS clients for the apps; `buf generate` → Go/Dart/Rust for the agent protobuf. **No hand-written clients**, so the three codebases can't drift (architecture §4.2).

---

9. `helixvpnctl` & Podman deployment

9.1 Bootstrap CLI (replaces the original doc's bash install scripts)

```
helixvpnctl init \
  --domain gw.example.com \
  --data-dir /var/lib/helixvpn        # generates: tenant, admin,
        CA root, overlay /48,
                                       # Postgres+Redis creds,
        gateway WG keys, quadlets
helixvpnctl gateway keys              # rotate gateway WG keys
helixvpnctl enroll-token --kind connector --site warehouse   #
        mint a token / QR
helixvpnctl policy apply ./policy.jsonc                       #
        validate + activate (GitOps-friendly)
helixvpnctl device revoke <id>
```

9.2 Podman quadlets (rootless, systemd-managed — no Docker daemon)

```
# /etc/containers/systemd/helixvpn.pod
[Pod]
PodName=helixvpn

# helixd.container — the control plane
[Container]
Image=ghcr.io/helixdevelopment/helixd:1.0
Pod=helixvpn
Environment=DATABASE_URL=postgres://helix@localhost/helix
Environment=REDIS_URL=redis://localhost:6379
# non-superuser DB role so RLS is enforced
```

```
# helix-edge.container — the data plane (winner of Phase-0 G4)
[Container]
Image=ghcr.io/helixdevelopment/helix-edge:1.0
Pod=helixvpn
AddCapability=NET_ADMIN
# :443/udp MASQUE + kernel WG; no SSH in this container
```

`postgres` and `redis` join the same pod; `helixvpnctl init` writes these unit files. Single-node self-host = `systemctl --user start helixvpn-pod`. The same images scale to the Phase-2 fleet by splitting the pod across hosts.

---

10. Reconciliation end-to-end (the MVP's signature flow)

```
admin: helixvpnctl policy apply  → REST POST /v1/policies
identity/policy: dry-run compile OK → persist version N → emit
policy.updated
policy svc: compile(N) → persist compiled → emit
policy.compiled{version:N}
coordinator: consume policy.compiled → recompute tenant visibility
   → for each affected open WatchNetworkMap stream: compute
MapDelta → Send
client/connector cores: receive delta → reconcile (add/remove WG
peers, update AllowedIPs)
edge: peer set + verdict map updated → enforcement live
  ── elapsed, event→enforced: target p99 < 1s, zero restarts ──
Console: live WS event "policy v N active; 7 devices updated"
```

This is the architecture doc's promise ("policy change reflected on all affected edges in < 1 s, no restarts") turned into a concrete, measurable MVP behavior — and it's a direct descendant of Phase 0's G6 (file-delta reconcile), now event-driven and authorized.

---

11. Testing & acceptance

11.1 Test layers

- **Unit:** policy compiler (table-driven: spec → expected visible/ allowedIPs), IPAM allocator, event idempotency.
- **Store:** RLS tests — assert tenant A literally cannot read tenant B's rows even with a crafted query.
- **Integration:** spin Postgres+Redis (testcontainers), drive enroll → advertise → policy → `WatchNetworkMap`, assert the delta stream.

- **End-to-end:** the Phase-0 netns rig, now fed by the real control plane instead of `map.json`; the §10 flow must converge < 1 s.
- **Soak:** N simulated agents holding streams; flap policies; assert convergence SLO and no memory growth in the coordinator.

11.2 MVP definition of done

1. Self-host from zero with `helixvpnctl init` + `systemctl start` on a fresh VPS.
2. Enroll a connector (advertises a LAN) and a client; client reaches an authorized LAN host; is denied an unauthorized one.
3. Auto transport ladder works: block plain WG → client escalates to MASQUE and stays up.
4. Edit policy via Console/CLI → affected devices reconfigured in < 1 s, no restart.
5. Revoke a device → loses access in < 1 s.
6. Kill-switch + DNS-leak protection verified (no plaintext egress when tunnel drops).
7. **No durable connection/traffic log exists** — verified by §11.4.
8. Access app (iOS/Android/Linux) + Connector daemon + Console (web) all drive the above.

11.3 SLOs (measured, alert if breached)

SLO	Target
event → delta-on-wire	p99 < 1 s
enroll → first NetworkMap	< 2 s
revoke → edge enforcement	< 1 s
coordinator memory @ 10k streams	bounded, no leak over 24h soak

11.4 No-logging CI lint

A schema test fails the build if a durable table named/scoped like a connection log appears (`connections`, `flows`, `traffic`, `packets`, or any table with both a `src`/`dst` and a `bytes`/`ts` column outside `audit_events`). The privacy promise is enforced by tooling, not trust.

---

12. What graduates to Phase 2

The MVP interfaces are built to extend without reshaping: `TransportPolicy.order` gains `shadowsocks`/`udp-over-tcp`/`daita`; `Peer.endpoint` gains direct-P2P candidates (NAT traversal) so

traffic stops always relaying through the gateway; `coordinator` federates across regional instances over NATS; `pki` adds the PQ pre-shared layer; the Connect service gains multi-hop path fields; and HarmonyOS/Aurora app shims slot under the unchanged Flutter UI + Rust core. Phase 2 is additive because Phase 1 drew the seams in the right places.

End of Phase 1 MVP build specification. The trilogy — Architecture (what), Phase 0 (prove the hard parts), Phase 1 (ship a self-hostable MVP) — is now complete and internally consistent: every Phase 1 interface is the durable, authorized evolution of a Phase 0 interface, and every Phase 1 component maps to a numbered section of the architecture.